Client-Side Scripting II

Presented by: Carlo Mamo

Bachelors of Science (Hons) (MQF Level 6)

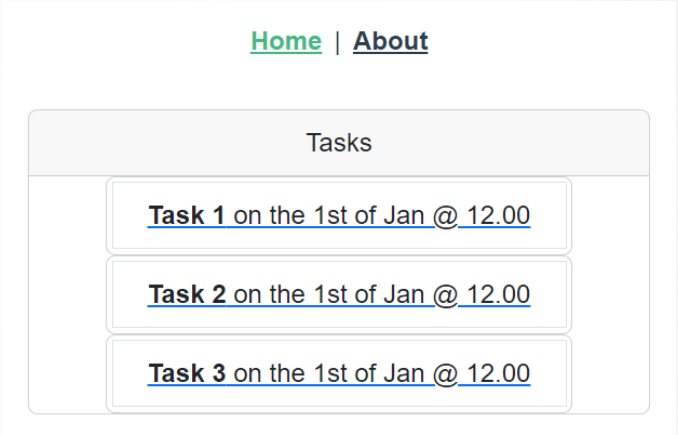
Chapter 4 – Routing and Pagination

Dynamic Routing

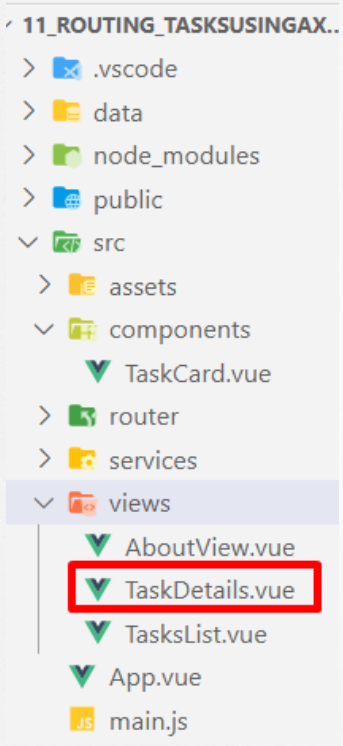
- In this chapter we will make all **tasks clickable** and once the user clicks a particular task all the details of that task will be shown.
- Hence we will need a new component showing all details (TaskDetails.vue) and will make an API call/request to fetch the data of **a single task using the id**.
- Once JSON response is received it will **be stored locally** and then displayed.

Exercise 1

- Create TaskDetails component
- Add new API call to fetch a single task by id
- Add new TaskDetails component to routes
- Make TaskCard clickable with router link



TaskDetails.vue




```
<script setup>
import TasksService from "@/services/TasksService.js";
import { ref, onMounted } from "vue";

const task = ref(null);

//import { useRoute } from 'vue-router'
//const route = useRoute()
//const id = ref(parseInt(route.params.id));
//we can either use route as shown above to get the id from
//the URL or else receive it as a prop as shown below.

const props = defineProps(["id"]);

onMounted(() => {
  TasksService.getTask(props.id)
    .then((response) => {
      task.value = response.data;
    })
    .catch((error) => {
      console.log("ERRRORS" + error);
    });
});
</script>
```

```
<template>
  <div v-if="task" class="task-details">
    <h1>{{ task.title }}</h1>
    <p>{{ task.time }} on {{ task.date }} @ {{ task.location }}</p>
    <p>{{ task.description }}</p>
  </div>
</template>
```

```
<style scoped>
.task-details {
  border: 1px solid ■■black;
  width: 100%;
  margin: 15px auto;
  padding: 10px;
}
</style>
```

Task 2

12.00 on 1st of Jan @ Valletta

Description for task 2

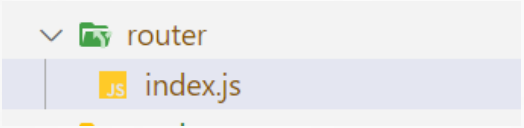
TasksService.js

- Adding the new API call to request one task

```
  getTasks () {
    return apiClient.get('/tasks')
  },
  getTask (id) {
    return apiClient.get('/tasks/' + id)
  }
}
```

Adding route and making task clickable

- To use props in TaskDetails we need to specify this in router.




```
{
  path: '/tasks/:id',
  name: 'TaskDetails',
  // so we can send in the route params as component props
  props: true,
  component: TaskDetails
}
```

Adding route and making task clickable

- To make the task clickable we need to add the following in TaskCard.vue and use router-link:

```
<template>
  <router-link
    class="task-link"
    :to="{ name: 'TaskDetails', params: { id: task.id } }"
  >
    <li class="list-group-item">
      <b>{{ task.title }}</b> on the {{ task.date }} @ {{ task.time }}
    </li>
  </router-link>
</template>
```

Making task id dynamic

- The task id needs to be added to the router's parameters. In TaskCard we have access to the id (params) through the prop.

```
<router-link :to="{name: 'TaskDetails', params:{id: task.id}}">
```

```
const props = defineProps({ task: Object });
```

Accessing route id

- To access the route id we have 2 options. We can either use props and do the following:

```
const props = defineProps(["id"]);

onMounted(() => {
  TasksService.getTask(props.id)
    .then((response) => {
      task.value = response.data;
    })
```

Accessing route id

- Or we can get the route id from the url by doing the following:

```
//import { useRoute } from 'vue-router'
//const route = useRoute()
//const id = ref(parseInt(route.params.id));
```

Possible issue

- It is possible that TaskDetails.vue tries to display a task but the API call hasn't yet returned its response and hence we don't have a task yet. Possibly the below error will be shown.

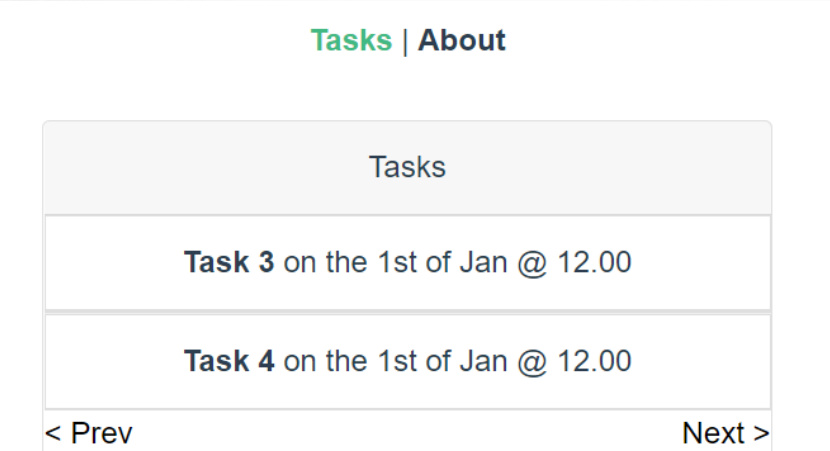
```
    at <App>
Uncaught (in promise) TypeError: Cannot read property 'title' of null
```

- To fix this error just add a conditional if statement where the TaskDetails div is displayed only when a task exists. This will make the component wait for the task to be retrieved before being displayed.

```
<div v-if="task" class="task-details">
```

Exercise 2 - Building Pagination

- As can be seen below a previous and next button have been added which when clicked loads up the tasks based on which page we are on.



Building pagination

- Step 1 – Modify TasksService API to take perPage and page parameters
- Step 2 - Parse and set the current page from the router
- Step 3 – Modify TaskList.vue to pass on page number to TasksService API
- Step 4 – Add pagination links to the TaskList template
- Step 5 – Only show a Previous/Next page link when there is a Previous/Next page
- Step 6 – Modify pagination styling

Building pagination

- The JSON Server uses the following end point: **http://localhost:3000/tasks**
- According to the server documentation a **_limit** parameter (how many items to return per page) and a **_page** parameter (which page to return) can be sent. E.g.

http://localhost:3000/tasks**?_page=1&_limit=2**

- The above url will return 2 tasks per page and will display page 1.

Building pagination

- We will start by adding 2 parameters to the getTasks method; **perPage** which is the number of tasks to return per page and **page** which is the page number we are on.

```
getTasks () {
  return apiClient.get('/tasks')
},
```

```
export default {
  getTasks(page, perPage) {
    return apiClient.get("/tasks?_page=" + page + "&_limit=" + perPage);
  },
};
```

Parsing and setting the current page

- If a query parameter exists in the url then we need to get the parameter and cast it to an integer.
- If it does not exist, we default it to page 1. The page parameter must be sent as a prop into TasksList.vue component.

```
routes: [
  {
    path: "/",
    name: "TasksList",
    component: TasksList,
    //if page exists parse the string to an integer otherwise return 1
    //When clicking on Tasks in the menu item, page 1 will be shown as no page query exists
    props: (route) => ({ page: parseInt(route.query.page) || 1 }),
  },
```

Modify TasksList.vue to pass on page

- The TaskList component must receive a prop called page which is then passed to the API method call together with the number of tasks shown per page.

```
<script setup>
// @ is an alias to /src
import TaskCard from "@/components/TaskCard.vue";
import TasksService from "@/services/TasksService.js";

import axios from "axios";
import { ref, onMounted, computed, watchEffect } from "vue";

const props = defineProps({
  page: Number,
});
```

```
onMounted(() => {
  watchEffect(() => {
    //TasksService.getTasks()
    TasksService.getTasks(props.page, 2)
      .then((response) => {
        tasks.value = response.data;
```

- Try typing in the following URL - http://127.0.0.1:5173/?page=2. Page 2 should be shown. The page number can be displayed in the console with - *console.log(props.page)*.

Adding pagination links

- Add links below.

```
<div class="pagination">
  <!--  adding Previous and Next Page links -->
  <router-link
    id="prev"
    :to="{ name: 'TasksList', query: { page: page - 1 } }"
    v-if="page != 1"
  >
    &#60; Prev
  </router-link>

  <router-link
    id="next"
    :to="{ name: 'TasksList', query: { page: page + 1 } }"
    v-if="hasNextPage"
  >
    Next &#62;
  </router-link>
</div>
```

- After adding the links above there is a problem. The components are not updating. This is happening because the onMounted() lifecycle hook is **only called when the component is initially loaded** and when we go to a new page **VUE is reusing the component and it does not call onMounted() again**.

Making another API call when page is updated

- The work around for this problem is to use the VUE 3 Composition API **watchEffect** function. (Link)

- The **watchEffect** method is used to track reactive dependency and run a method when reactive objects that are accessed inside a function change. In our case **props.page** is being accessed and is reactive. Hence when page is changed the API is called again.

```
onMounted(() => {
  watchEffect(() => {
    //TasksService.getTasks()
    TasksService.getTasks(props.page, 2)
      .then((response) => {
        tasks.value = response.data;
      })
      .catch((error) => {
        console.log("ERRRORS" + error);
      });
  });
});
```

Only show Next link if there is a next page

- To show the Next link only if there is a next page we need to know the total number of tasks. We can get this from a header that contains this information which is returned by the API.
- Add a property **totalTasks** TasksList.vue

```
const totalTasks = ref(0);
```

- After returning the data from the API call set the totalTasks property

```
tasks.value = response.data;
totalTasks.value = response.headers["x-total-count"];
```

- Then use the above information inside a computed property.

Only show Next link if there is a next page

```
const hasNextPage = computed(() => {
  //  Math.ceil - rounds up the value to the nearest larger number.
  //  divide by 2 since 2 tasks per page are being shown
  var totalPages = Math.ceil(totalTasks.value / 2);
  //  if this page is not the last page computed property returns true
  return props.page < totalPages;
});
```

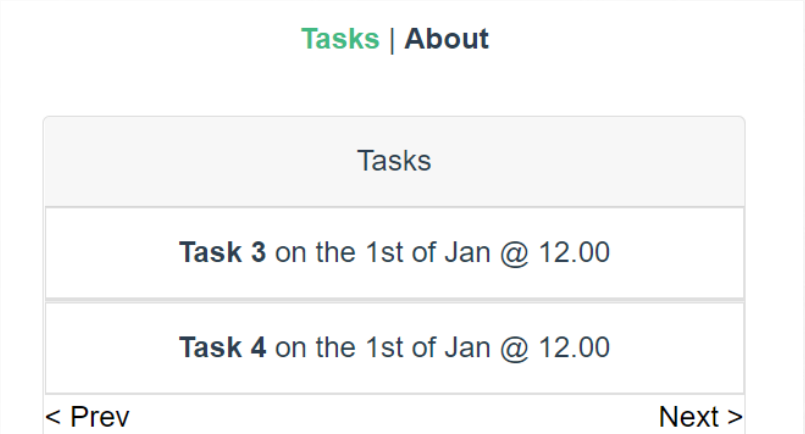
- Only display Next Page if – hasNextPage returns true. Make sure that when you are on the last page Next Page disappears.

```
<router-link
  :to="{ name: 'TasksList', query: { page: page + 1}}"
  v-if="hasNextPage"> Next Page
</router-link>
```

Styling

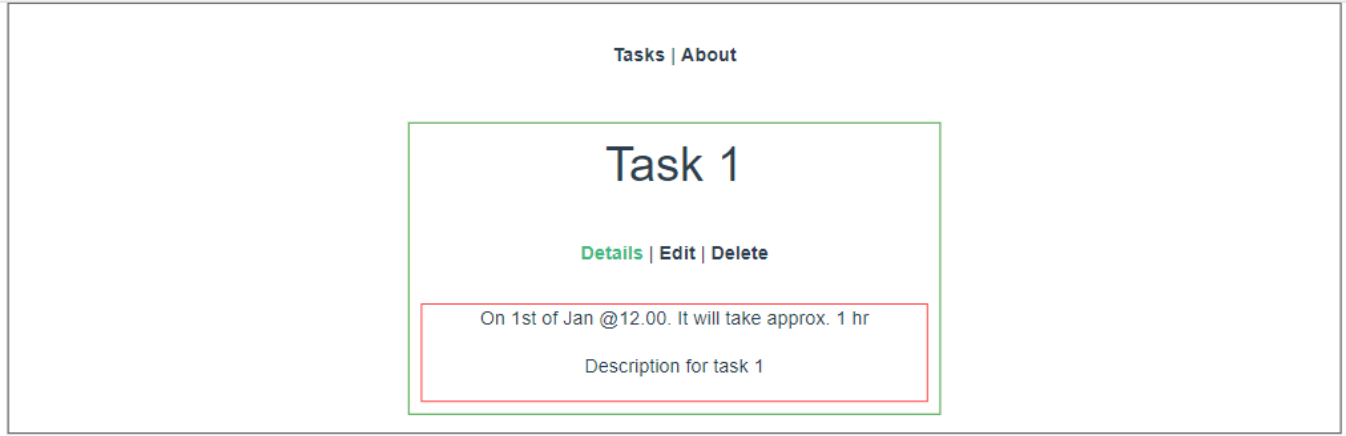
- Add a pagination class to the Previous and Next links. Add an id to both the prev and next links.

```
.pagination{
  display: flex;
  width: 382px;
}

.pagination a{
  flex: 1;
  text-decoration: none;
  color: black
}

#prev{
  text-align: left;
}

#next{
  text-align: right;
}
```



Adding Nested Routing – Exercise 3

- As shown below we will now add: Details, Edit and Delete tabs for each Task.



Nested Routes

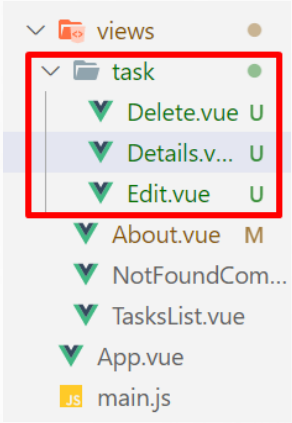
- If we want to have good URL design for the below actions we would have the following:

ACTION	URL	LOCATION
Task Details	/task/2	/src/views/task/Details.vue
Edit task	/task/2/edit	/src/views/task/Edit.vue
Delete task	/task/2/delete	/src/views/task/Delete.vue

- How to create a route for these views?

Nested Routes

- If we use Normal routing this will cause some problems due to code duplication. Hence the VUE router feature Nested routes will be used.
- Create a new folder task in the views folder. In this folder add the TaskDetails.vue file and rename it to Details.vue.
- Create another 2 files: Edit.vue and Delete.vue



Nested Routes

- In Details.vue add the navigation as shown below.

```
<template>
  <div v-if="task" class="task-details">
    <h1>{{task.title}}</h1>
    <div id="nav">
      <router-link :to="{name: 'TaskDetails', params: {id}}">Details</router-link> |
      <router-link :to="{name: 'TaskEdit', params: {id}}">Edit</router-link> |
      <router-link :to="{name: 'TaskDelete', params: {id}}">Delete</router-link>
    </div>
    <p>{{task.time}} on {{task.date}} @ {{task.location}}</p>
    <p>{{task.description}}</p>
  </div>
</template>
```

Nested Routes

- Copy all code from Details.vue and paste in Edit.vue and Delete.vue. Just change the content as follows:

```
<template>
  <div v-if="task" class="task-details">
    <h1>{{task.title}}</h1>
    <div id="nav">
      <router-link :to="{name: 'Details', params: {id}}">Details<router-link> |
      <router-link :to="{name: 'Edit', params: {id}}">Edit<router-link> |
      <router-link :to="{name: 'Delete', params: {id}}">Delete<router-link>
    </div>
    <p>Delete task</p>
  </div>
</template>
```

```
<template>
  <div v-if="task" class="task-details">
    <h1>{{task.title}}</h1>
    <div id="nav">
      <router-link :to="{name: 'Details', params: {id}}">Details<router-link> |
      <router-link :to="{name: 'Edit', params: {id}}">Edit<router-link> |
      <router-link :to="{name: 'Delete', params: {id}}">Delete<router-link>
    </div>
    <p>Edit task</p>
  </div>
</template>
```

Nested Routes

- In /router/index.js we need to first change the name for the Details.vue and then add import statements for both Edit.vue and Delete.vue. Add paths to edit and delete as shown below.

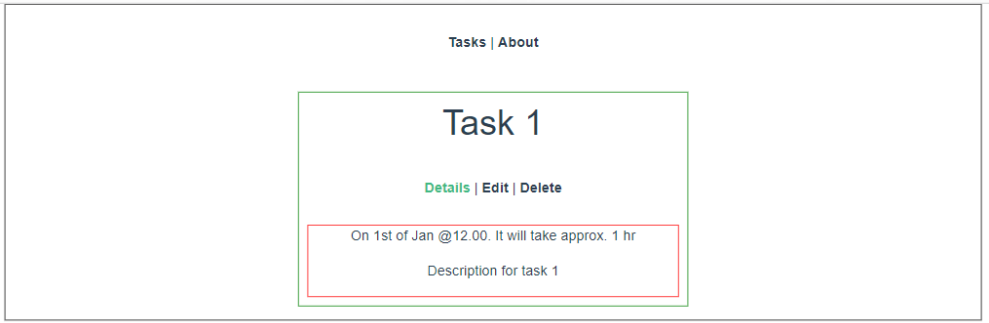
```
{
  path: '/tasks/:id',
  name: 'TaskDetails',
  //  so we can send in the route p
  props: true,
  component: TaskDetails
},
{
  path: '/tasks/:id/edit',
  name: 'TaskEdit',
  props: true,
  component: TaskEdit
},
{
  path: '/tasks/:id/delete',
  name: 'TaskDelete',
  props: true,
  component: TaskDelete
},
```

Nested Routes – Exercise 4

- Is there duplicate code that you think we can get rid off?
- What about the navigation links and calling the API?
- We will try and solve this by using nested routes.

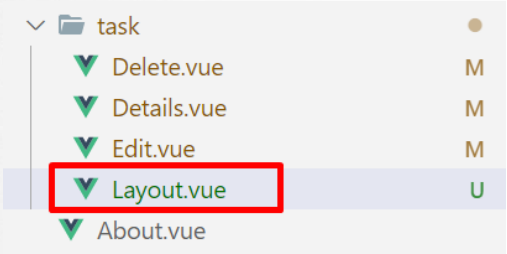
Creating a Layout component

- This layout component will contain the navigation for each task page and the API call for fetching the task (src/views/Layout.vue). Layout in figure below has a green border.
- After fetching the task this will be sent to the children components i.e. Details.vue (red border), Edit.vue and Delete.vue.
- Its important to mention that we already have a base layout in the App.vue (black border).



Layout.vue component

- Copy all code from Details.vue and paste in Layout.vue. Remove details and replace with a router-view directive and send task that we fetch from the API.




```
<template>
  <div v-if="task" class="task-details">
    <h1>{{task.title}}</h1>
    <div id="nav">
      <router-link :to="{name: 'TaskDetails', params: {id}}">Details</router-link> |
      <router-link :to="{name: 'TaskEdit', params: {id}}">Edit</router-link> |
      <router-link :to="{name: 'TaskDelete', params: {id}}">Delete</router-link>
    </div>
<!-- sending task which is fetched from API below -->
<router-view :task="task" /> <!-- this is where the children components will be displayed on screen -
  </div>
</template>
```

Children components

- We can delete everything from the children components except from receiving the task prop.

```
<template>
  <p>
    On {{ task.date }} @{{ task.time }}. It will take approx.
    {{ task.duration }}
  </p>
  <p>{{ task.description }}</p>
</template>

<script setup>
const props = defineProps({
  task: Object,
});
</script>
```

```
<template>
  <p>Edit</p>
</template>

<script setup>
const props = defineProps({
  task: Object,
});
</script>
```

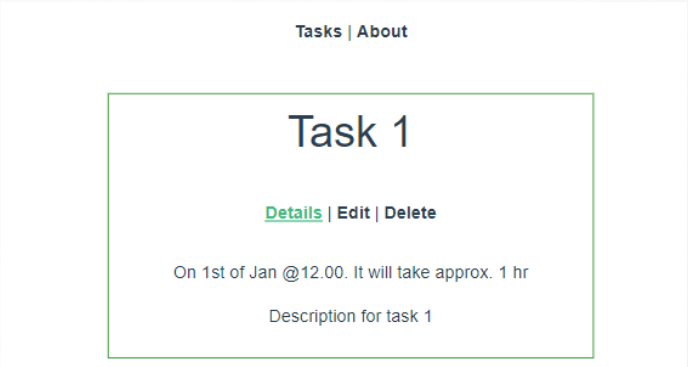
```
<template>
  <p>Delete</p>
</template>

<script setup>
const props = defineProps({
  task: Object,
});
</script>
```

Index.js in router folder

- A children array is added and the nested child components will be added here.

```
{
  path: '/tasks/:id',
  name: 'TaskLayout',
  props: true,
  component: TaskLayout,
  children: [
    { // nest child components
      path: '', // this will be loaded at the root path of the parent
      name: 'TaskDetails',
      component: TaskDetails
    },
    {
      path: 'edit', // this will be added to the parent path
      name: 'TaskEdit',
      component: TaskEdit
    },
    {
      path: 'delete', // this will be added to the parent path
      name: 'TaskDelete',
      component: TaskDelete
    }
  ]
},
```

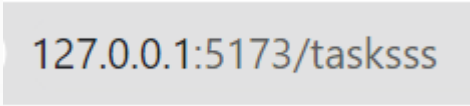
We will have the same functionality but with less duplicate code.



Cath-all route

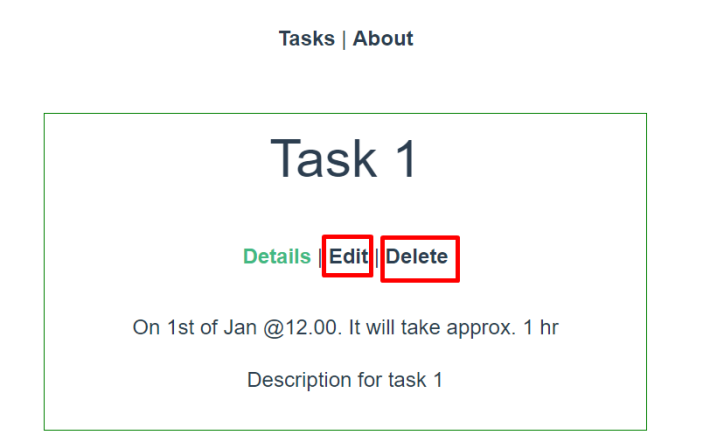
- We can add a catch-all route in router/index.js so that if the user tries to access an invalid URL a component is used to show the user that the path does not exist.

```
{
  path: '/:catchAll(.*)',
  component: NotFoundComponent
}
```



Programmatic Navigation – Exercise 5

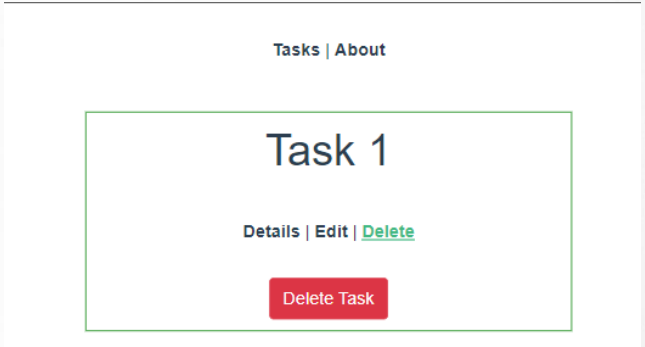
- Eventually we will want to create an edit form and a delete button in the edit and delete pages respectively. For now lets create a delete button which when clicked programmatically it navigate back to the task page.



Programmatic Navigation

- Add a button as shown below and once the user clicks the button the method deleteTask should be called. This method should only display on the console “Task deleted” and then redirect the user using the push method to the task details component. (Task will obviously not be deleted from the json file).

```
<script setup>
import { useRouter, useRoute } from "vue-router";

const props = defineProps({
  task: Object,
});

const router = useRouter();

function deleteTask() {
  console.log("Task deleted (Not from json file)");
  router.push({ name: "TaskDetails" });
}
</script>
```



Programmatic Navigation

- It's good to know that when calling **<router-link>** this is actually calling router.push from inside the router-link definition.
- For exa

```
<router-link :to="{name: 'About'}">About</router-link>
```

- When this link is clicked it calls: **router.push({ name: 'About' })**

Programmatic Navigation

Examples	
Path as a string	router.push({ '/about' })
Path with an object	router.push({ path: '/about' })
Named path	router.push({ name: '/about' })
Adding a dynamic segment	router.push({ name: '/taskDetails', params: { id: 3} })
Adding a query parameter	router.push({ name: '/taskList', query: { page: 2} })

The query example above will push to: /?page=2

Navigating the History stack

- To programmatically invoke the browser back and forward buttons use:

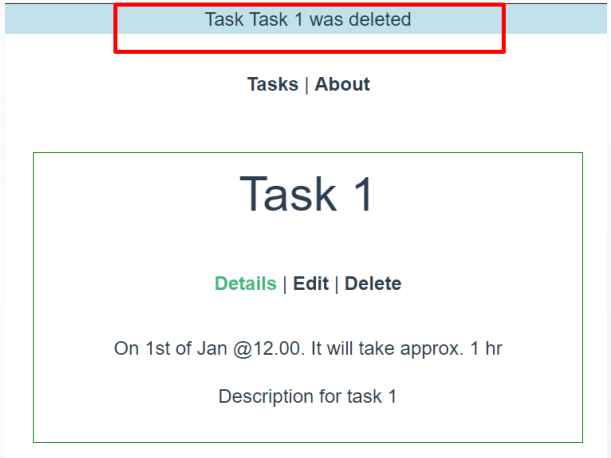
```
router.go(1) // to go forward like the browser's forward button

router.go(-1) //  to go backward like your browser's back button
```

- The above code may be useful for building a native app where there is no browser forward and back button.

Adding notification messages – Exercise 6

- When deleting a task ideally a notification is shown on screen confirming that a task was deleted.
- In the delete component we are currently assuming the API call has successfully deleted a task and then pushing the navigation to the TaskDetails component. A success message should be shown before the navigation is pushed.

```
function deleteTask() {
  console.log("Task deleted (Not from json file)");
  //flash message goes here
  router.push({ name: "TaskDetails" });
}
```



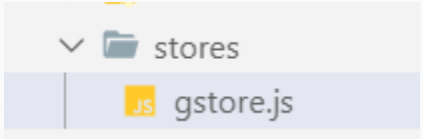
Adding notification messages

- To add a notification the following must be done:

1. Add Pinia – npm i pinia
2. Add Pinia in main.js (Refer to Example 05_storePiniaEx1)
3. Create a folder stores and a file within this folder gstore.js
4. Within the state create a flash message and initialise to an empty string.
5. Set the flash message to *“Task 1 was deleted”* inside Delete.vue
6. Create a layout inside Ap.vue where flash message is displayed

Creating the state

- Add flashMessage in state and initiliase to an empty string.
- Also add an action (method) to update flashMessage when task is deleted. This should accept the updated text as an argument.



Set the flashMessage inside Delete.vue

```
function deleteTask() {
  console.log("Task deleted (Not from json file)");
  //flash message goes here
  GStore.updateGStore(props.task.title + " was deleted");
  //reset flash message after 3 seconds
  setTimeout(() => {
    GStore.updateGStore("");
  }, 3000);
```

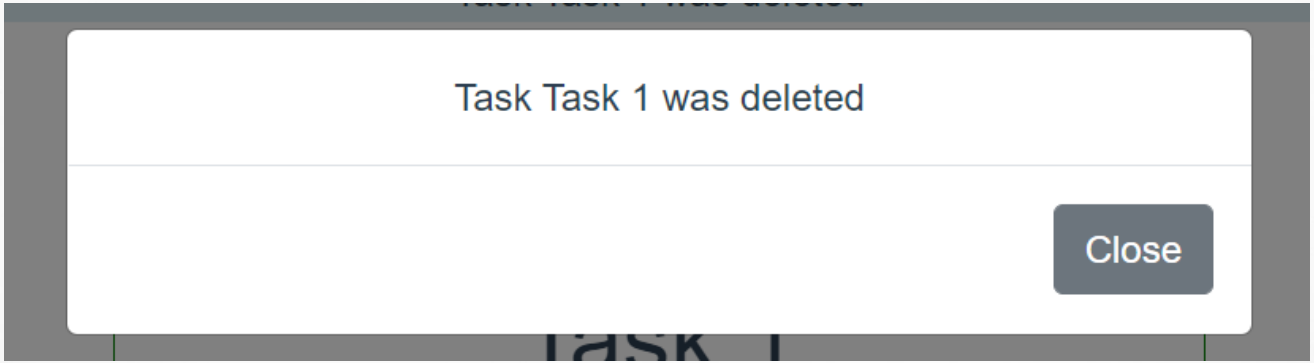
Set the layout in App.vue

```vue
<template>
  <div>
    <div id="flashMessage" v-if="GStore.flashMessage">
      {{ GStore.flashMessage }}
    </div>
    <nav>
      <router-link to="/">Home</router-link> |
      <router-link to="/about">About</router-link>
    </nav>
    <router-view />
  </div>
</template>

<script setup>
import { useGStore } from "@/stores/gstore";
const GStore = useGStore();
</script>
```

```css
@keyframes bluefade {
  from {
    background: lightblue;
  }
  to {
    background: transparent;
  }
}

#flashMessage {
  animation-name: bluefade;
  animation-duration: 3s;
}
```

Exercise 7

- Instead of showing a flash message show a modal when the user clicks the delete button as shown below. The user must then click the close button to hide the modal. (Must add code to the button in Delete.vue and App.vue)



References

Vuejs.org. 2021. *Vue.js*. [online] Available at: <https://vuejs.org/> [Accessed 7 February 2021].

Schweiger, P., 2021. *Vue Mastery*. [online] Vue Mastery. Available at: <https://www.vuemastery.com/> [Accessed 7 February 2021].

Vue School. 2021. *Learn Vue.js from core-team members and industry experts at Vue School*. [online] Available at: <https://vueschool.io/> [Accessed 7 February 2021].

Udemy. 2021. *Develop with VueJS 2 (Complete Vue.js Router and Vuex Course)*. [online] Available at: <https://www.udemy.com/course/vuejs-2-the-complete-guide/> [Accessed 7 February 2021].

Any questions?

THANK YOU

Email: carlo.mamo@mcast.edu.mt